Кейсы

Бронирования переговорных комнат

- Команда из трёх разработчиков делает приложение для бронирования переговорных комнат в корпоративных офисах.
  - Один из разработчиков предлагает сразу строить на микросервисах: «Если нас купят, нам придётся масштабироваться». Второй говорит: «Давайте сделаем монолит на Spring Boot за неделю».
- Функционал MVP: авторизация, просмотр и бронирование комнат, уведомления по email, простая аналитика (сколько раз забронировали за день).
- Какую архитектуру выбрать и почему?

Информационная система учёта транспорта

- Министерство транспорта заказывает АИС учёта подвижного состава. Требования зафиксированы в 400-страничном ТЗ, сроки - 2 года, заказчик - государство.
  - Ожидаемая нагрузка - 500 одновременных пользователей (диспетчеры, инспекторы)
  - Развёртывание - на серверах заказчика без доступа в интернет, поддержка - силами IT-отдела из 7 человек
- Архитектор предлагает микросервисы. Руководитель проекта - монолит.

- Кто прав?

Образовательная платформа с «уникальными» требованиями

- EdTech-компания разрабатывает платформу для онлайн-обучения. Команда - 8 разработчиков.
  - CTO классический евангелист микросервисов, прочитал книгу Сэма Ньюмана. Система разбита на 23 микросервиса: отдельный сервис для профилей пользователей, отдельный - для настроек профиля, отдельный - для аватаров, отдельный - для курсов, ещё один - для уроков внутри курсов, и так далее.
- Через полгода ☺ - время отклика страницы курса - 3–5 секунд (8 последовательных HTTP-вызовов между сервисами). Команда тратит 60% времени на отладку межсервисного взаимодействия. Новый разработчик не может поднять локально всю систему - это занимает 2 дня.
- В чём архитектурная ошибка?

Медицинская информационная система

- Клиника автоматизирует работу - запись пациентов, ведение электронных карт, расписание врачей, выписка рецептов, биллинг. Требования к надёжности высочайшие - сбой системы во время приёма недопустим. Данные критичны, требуется строгая транзакционная целостность (например, выписать рецепт и одновременно списать препарат со склада — это должно быть атомарно).
- Команда: 6 разработчиков. Заказчик ожидает систему на 10 лет.
- Архитектор предлагает микросервисы «ради отказоустойчивости»

- Ваше мнение?

Тот самый монолит

- Крупная логистическая компания. Монолит написан 8 лет назад, работает, приносит деньги. Технический долг большой, но система стабильна.
- Новый CTO приходит с мандатом «цифровой трансформации» и объявляет: «Переписываем всё с нуля на микросервисы за год».

- Ваше мнение?

Монолит

Монолит

- Что такое монолит
- Слои приложения
- Как выглядят монолиты в архитектуре
- Основные проблемы монолитов
- Когда полезен монолит
- Анализ монолита

Монолит

- Монолит - это одно приложение
- Монолит - это одно приложение созданное для уборщика, менеджера, поста охраны, администратора и гостя отеля
- Монолит - содержит в себе код пользовательского интерфейса и код доступа к данным и бизнес-логику
- Монолит - может быть многопоточным или однопоточным

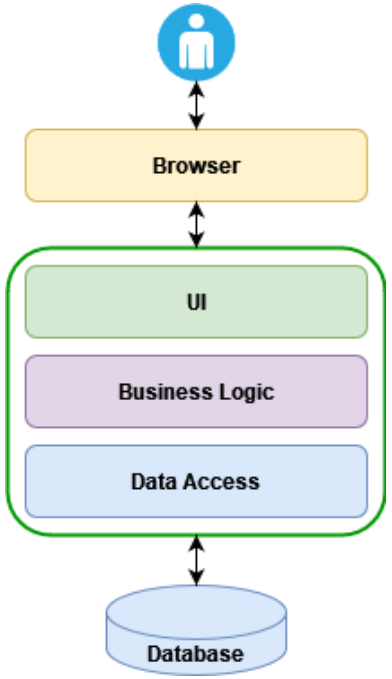
Монолит

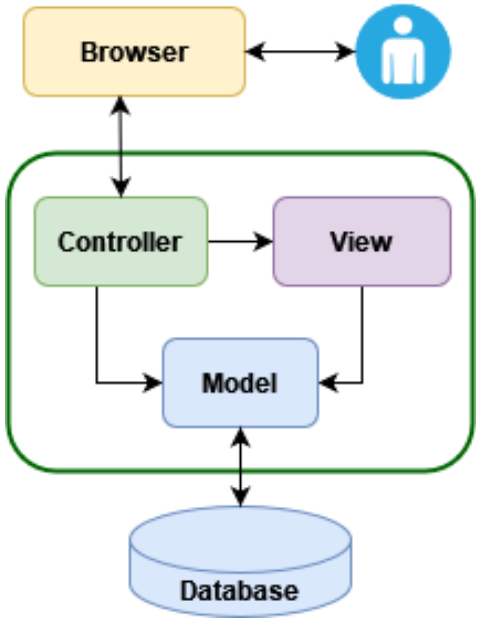
- Монолит - может быть модульным, одним приложением или связанной группой приложений
- Монолит - можно засунуть в Docker-контейнер или VM, Bare metal
- Монолит - единая кодовая база для всего проекта, нельзя просто взять и выделить отдельный код и запустить параллельно от монолита
- Монолит - когда для обновления нужно сначала заглушить двигатель Монолит - это иногда одна БД для всех приложений

Слои приложения

- Пользовательский интерфейс (UI)
- Слой бизнес логики (Business Logic)
- Слой доступа к данным (Data Access)

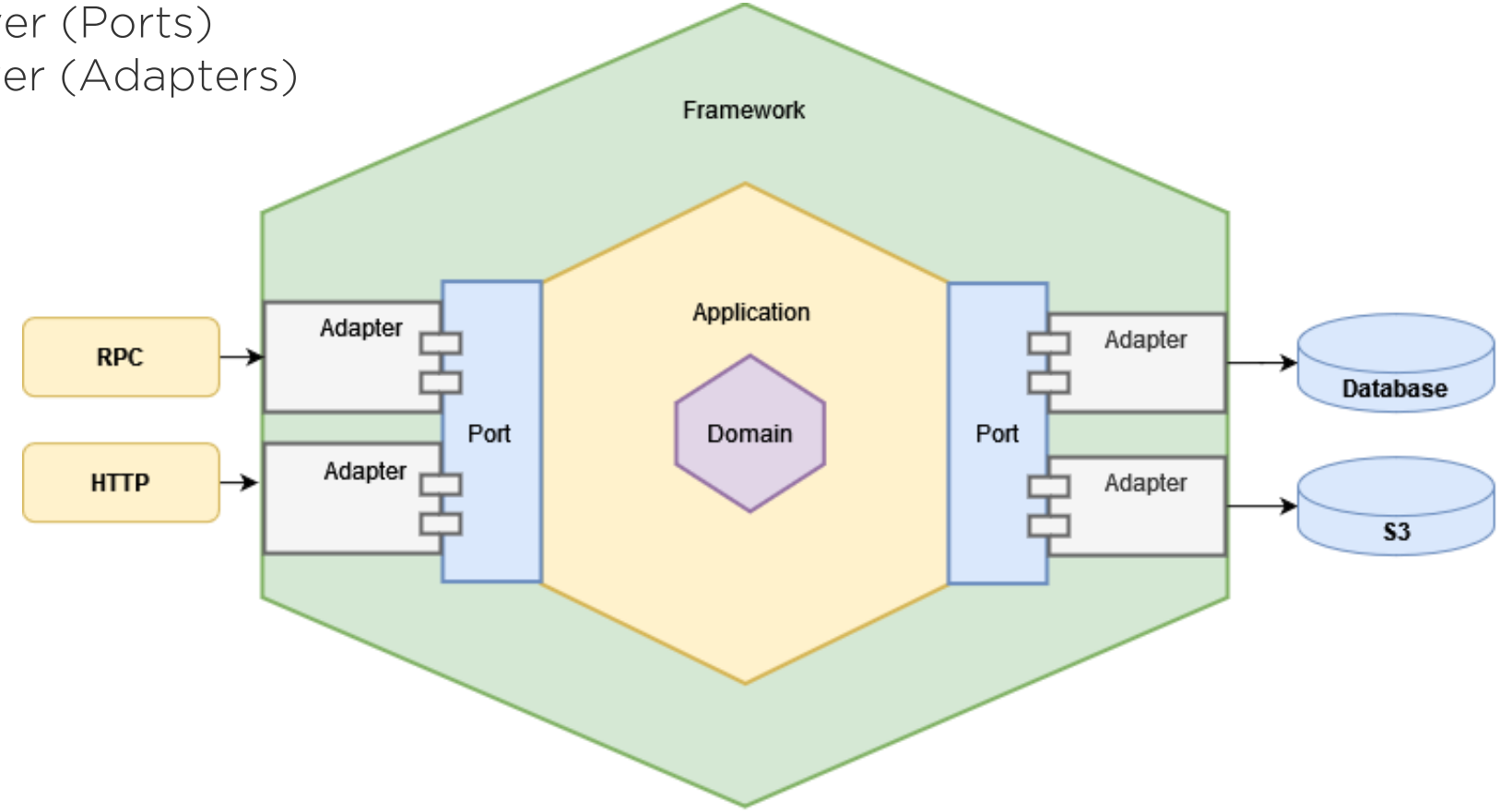
- View
- Controller
- Model





Слои приложения

- Domain Layer
- Application Layer (Ports)
- Framework Layer (Adapters)

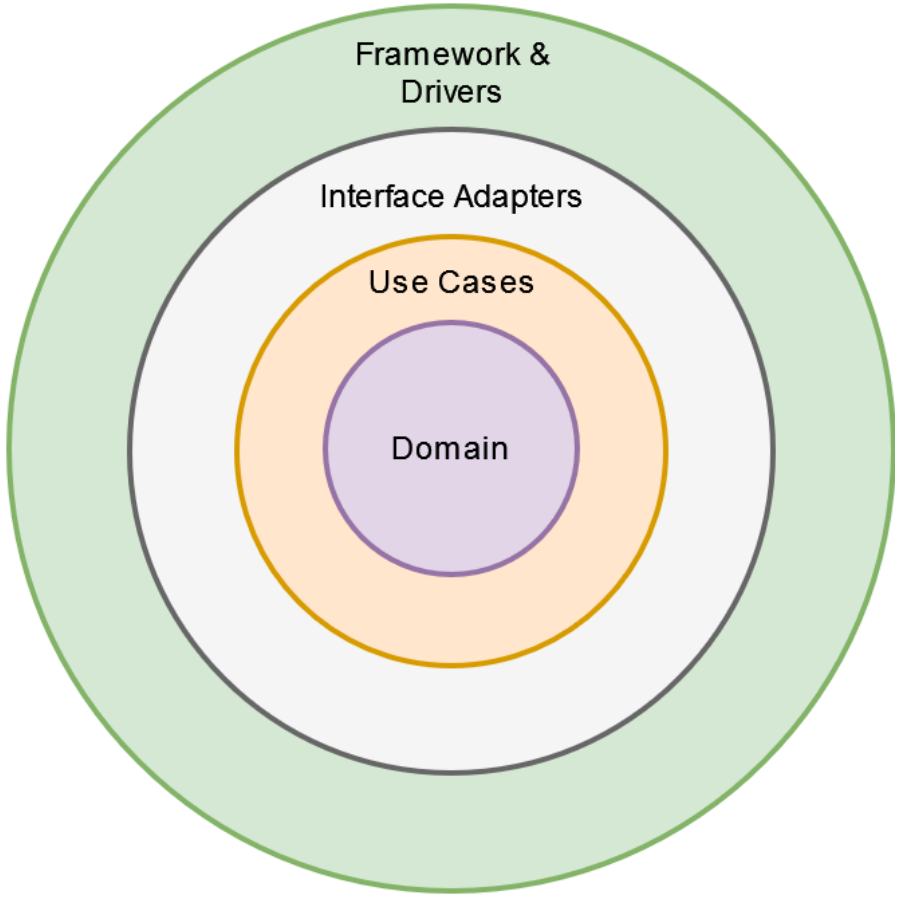


Слои приложения

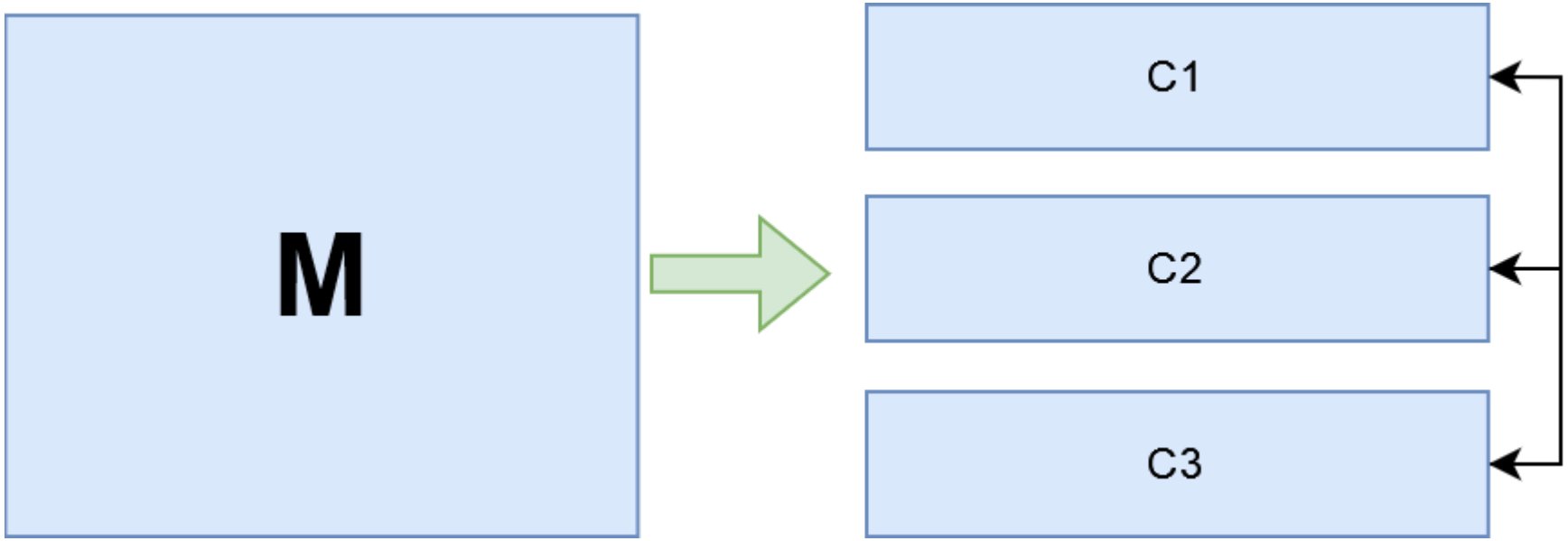
- Domain - Entities
- Use Cases - Use Cases
- Interface Adapters - Controller
- Framework & Drivers

- Framework & Drivers Initialize
  - Infrastructure Adapter
  - Interface Adapter
- Всё то что обеспечивает запуск приложения

- Interface Adapters Implements
  - Infrastructure Adapter
  - Interface Adapter
- Всё то что обеспечивает выполнение функций приложения



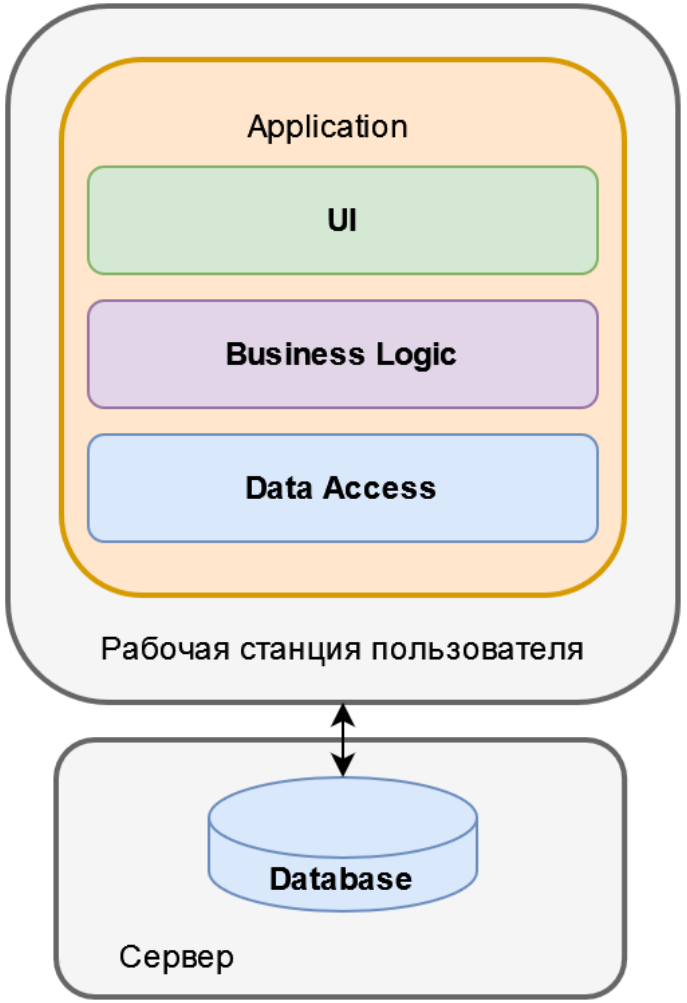
Размышление. Разделим монолит.



Как выглядят монолиты в архитектуре

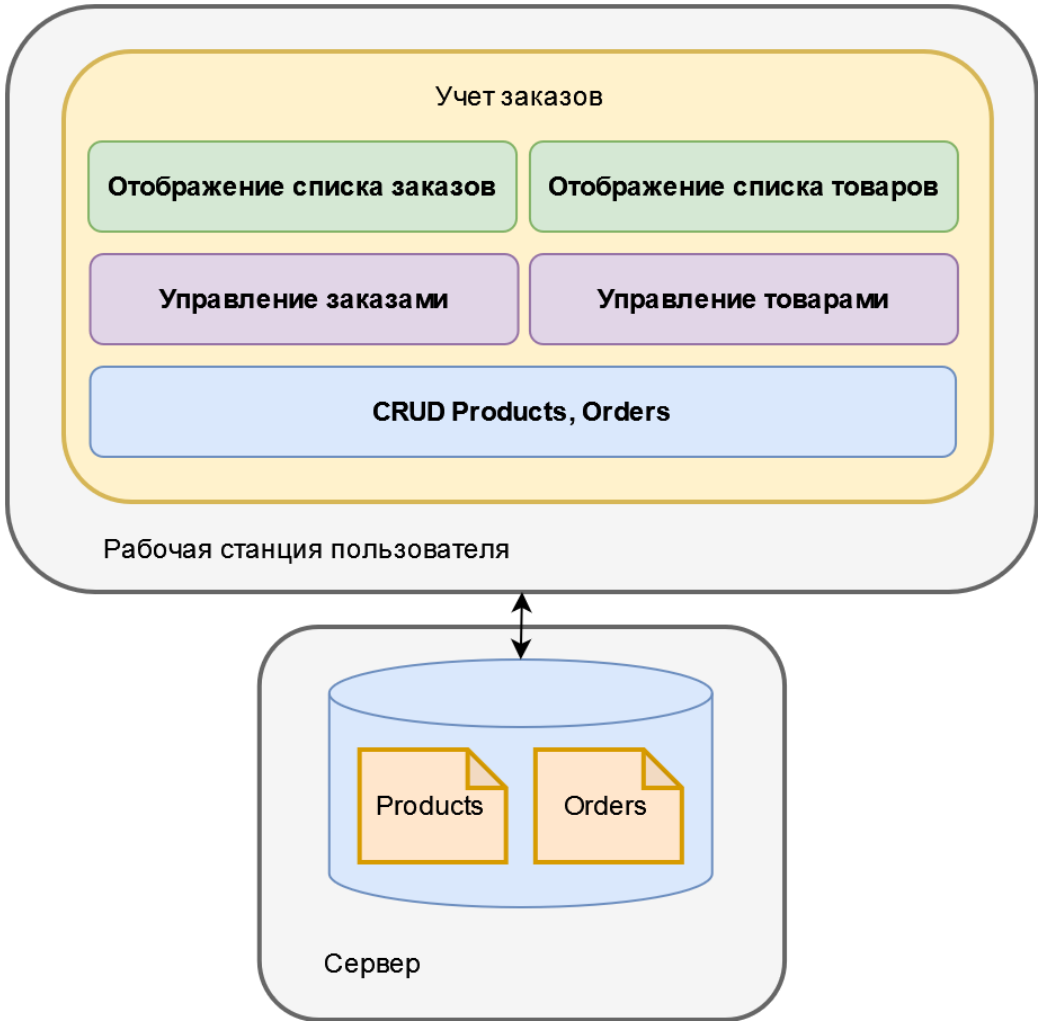
1. Находится на машине пользователя
2. Работает с удаленной базой
3. Бизнес логика покрывает множество доменов

Толстый клиент



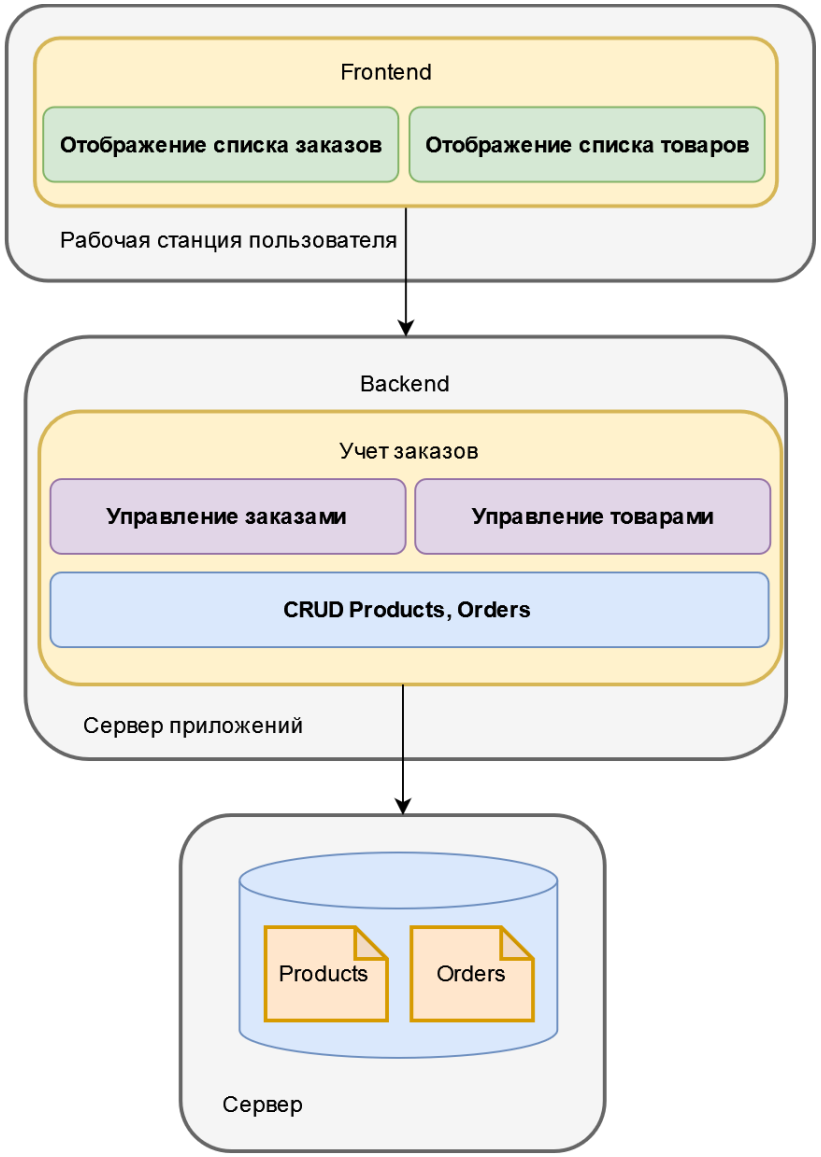
Как выглядят монолиты в архитектуре

- АРМ - Автоматизированное рабочее место
- АИС - автоматизированная информационная система



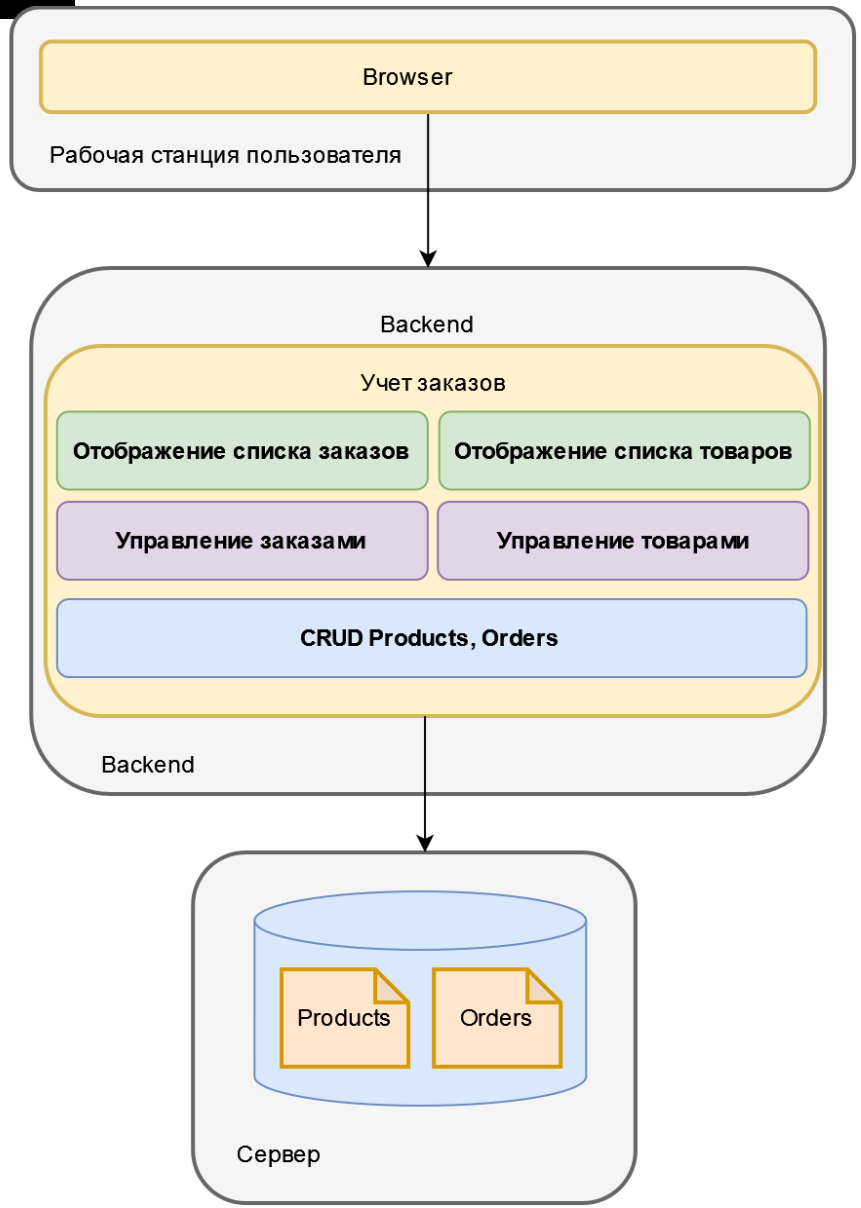
Как выглядят монолиты в архитектуре

- 3-х уровневая архитектура - архитектурная модель ПО, состоящая из трёх типов компонентов (уровней): клиентское приложение (с которыми работают пользователи), сервер приложений (с которым работает клиентское приложение) и сервер базы данных (с которым работает сервер приложений)
- Тонкий клиент, CSR - client side rendering

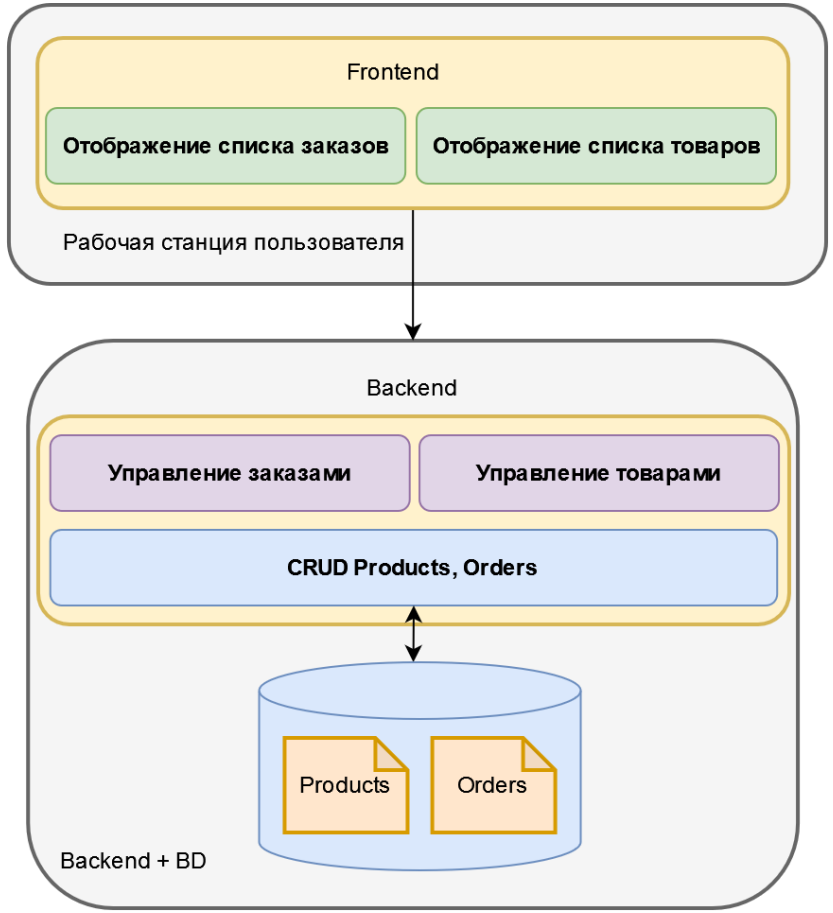


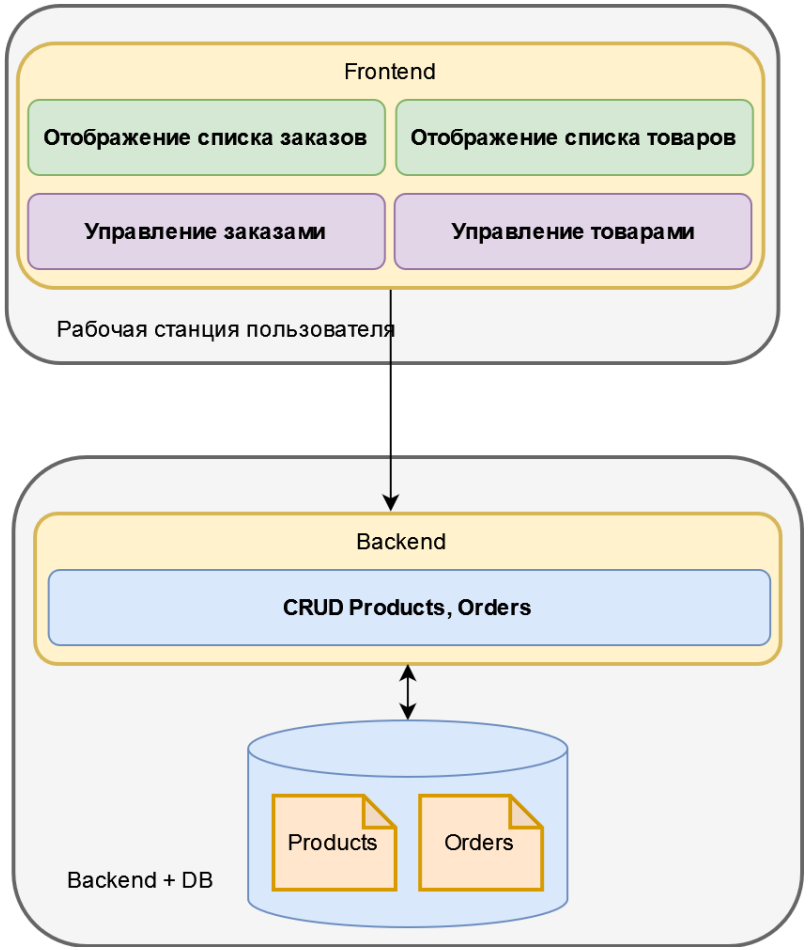
Как выглядят монолиты в архитектуре

- Выглядит как 3-х уровневая архитектура
- Работает как Тонкий клиент
- В результате SSR - server side rendering

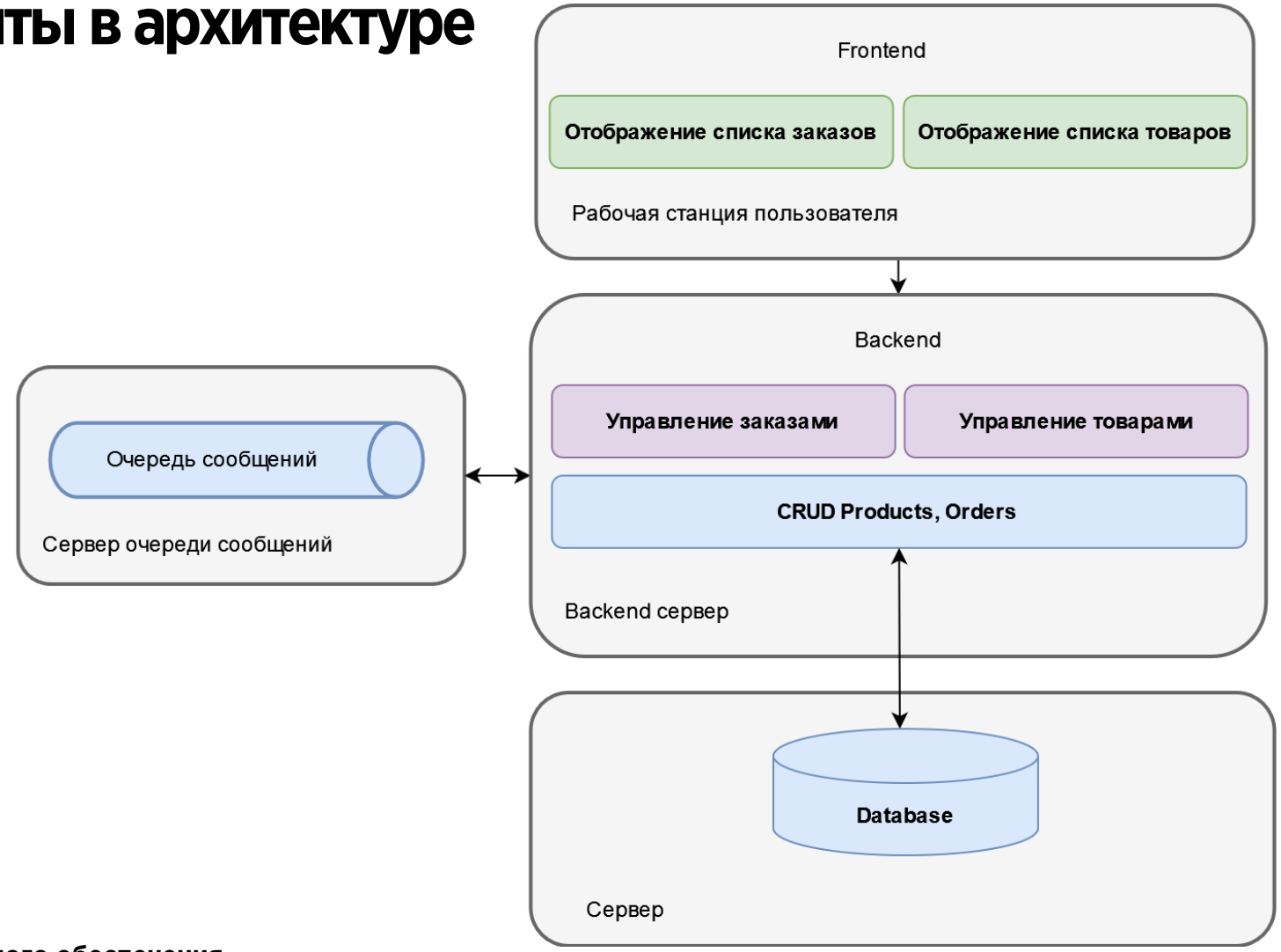


Как выглядят монолиты в архитектуре

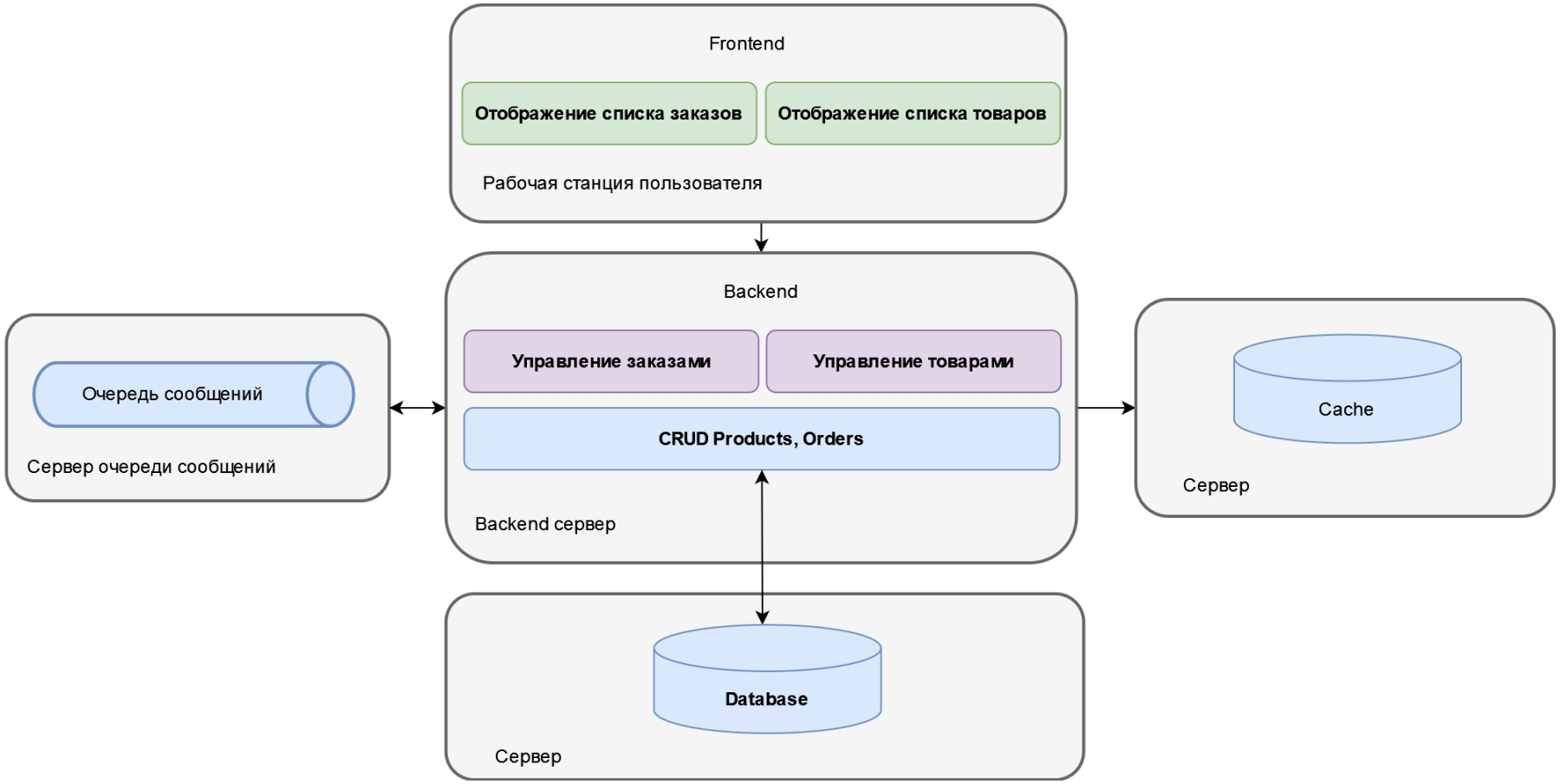




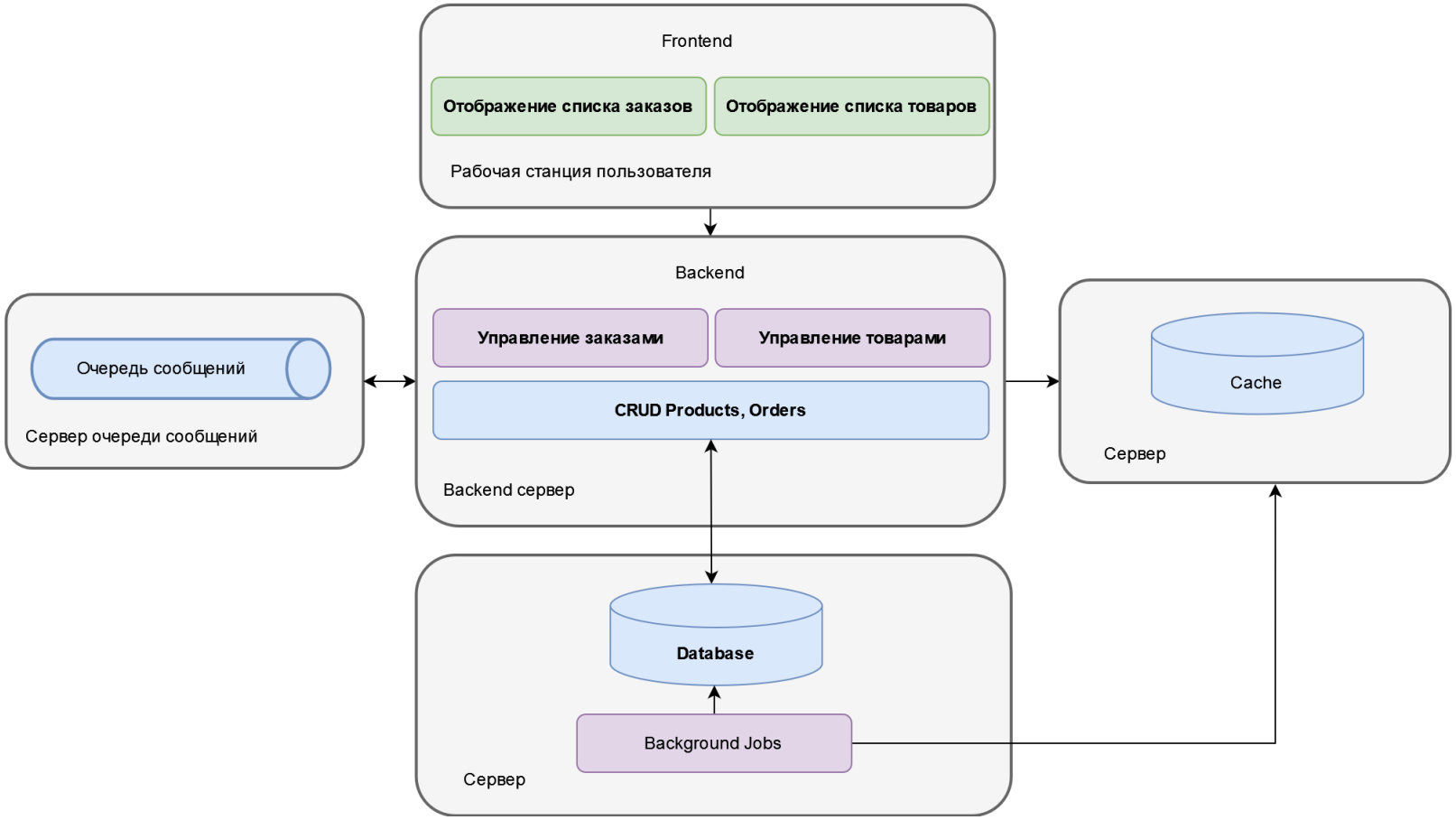
Как выглядят монолиты в архитектуре



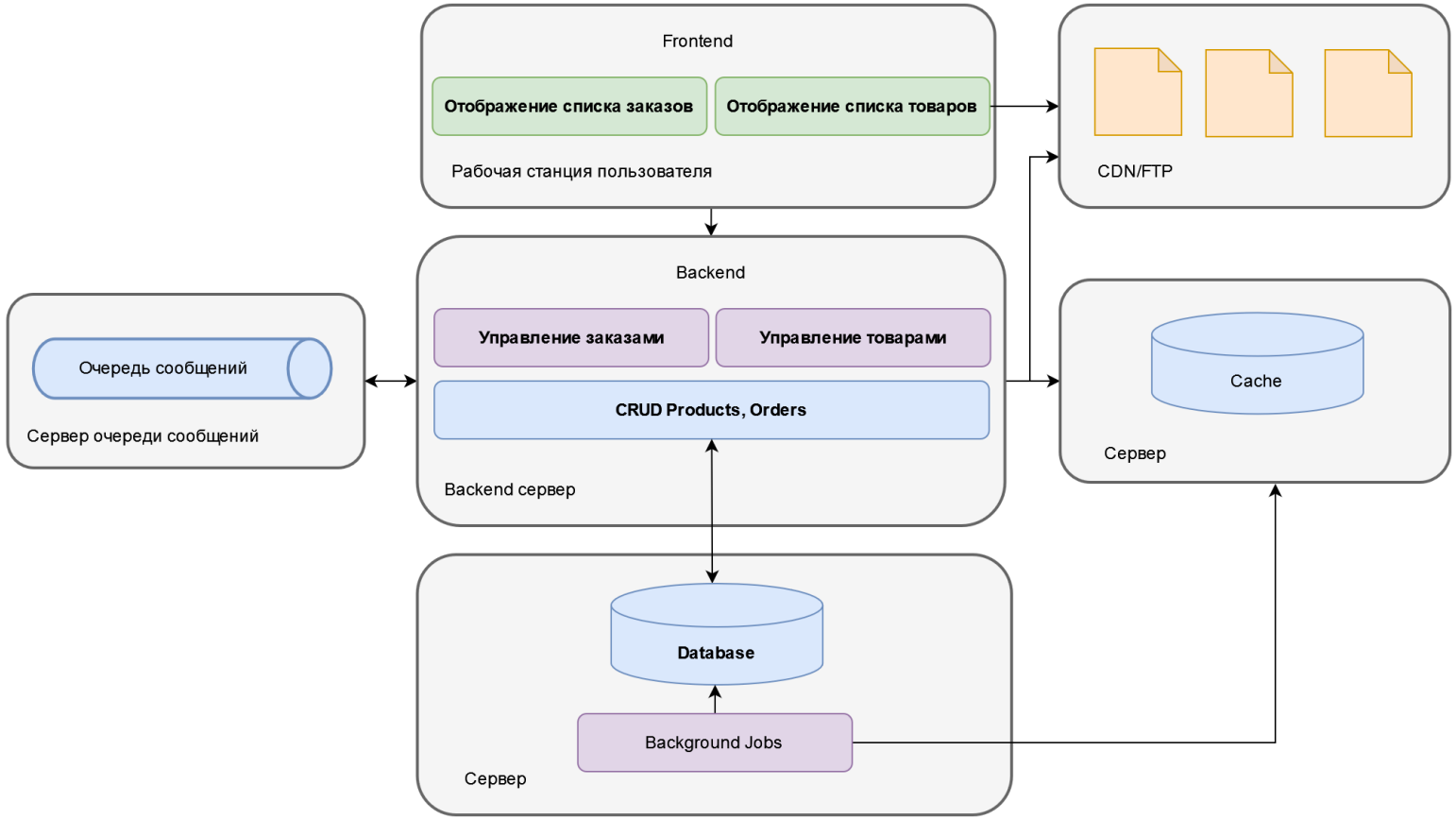
Как выглядят монолиты в архитектуре



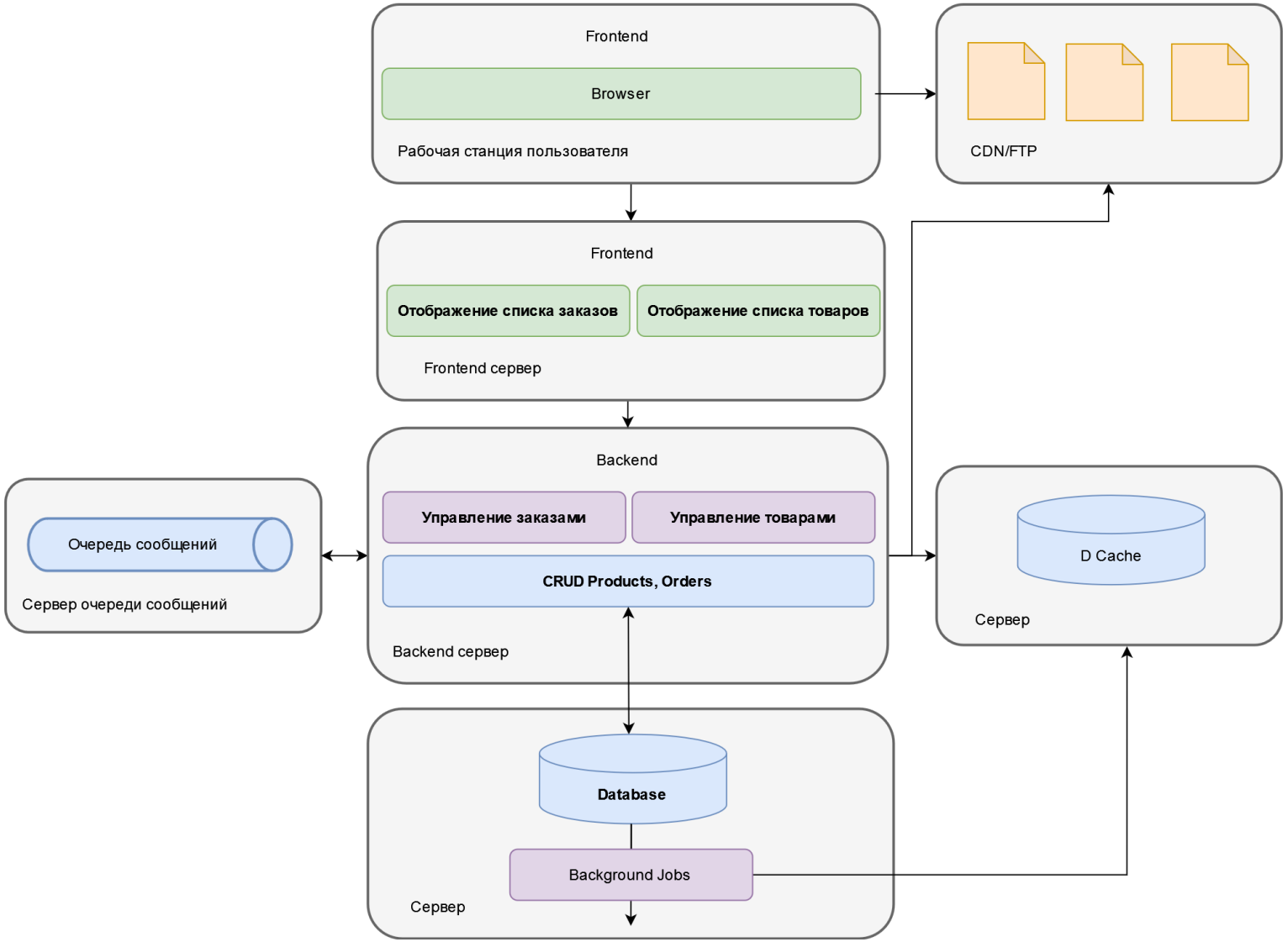
Как выглядят монолиты в архитектуре



Как выглядят монолиты в архитектуре



Микросервисная архитектура **Монолит**



Основные проблемы монолитов

- Не Гибкость
- Сложность Масштабирования
- Трудность в Тестировании
- Тяжёлая Отладка
- Долгая Доставка (Публикация), и с остановкой
- Не Устойчивость к падениям
- Плохая Доступность
- Плохая Отказоустойчивость

Когда монолит полезен

- Когда вы пишете десктопное приложение и оно должно работать в offline
- Когда мы не до конца понимаем каким должен быть продукт на выходе и используем подход MonolithFirst, со следующих точек зрения:
- Бизнес. Функциональное наполнение
- Поддержка. Условия эксплуатации
- Скорости вывода продукта на рынок (StartUp)

MonolithFirst

- Позволяет понять границы компонентов и их наполнение
- Показывает узкие места приложения
- Помогает понять общие микросервисы или Core
- В конечном итоге разбивается на микросервисы

Будут вопросы? Не стесняйтесь!